

Generative AI

★

Member-only story

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI coding updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.co

The EveryDESIGN.md Needs

CLAUDE.md taught my agent how to code. DESIGN.md stopped it from building ugly UI.



Yanli Liu

Follow

14 min read · May 6, 2026

👏 634

💬 3

↺ 1

🔖

▶

📄

⋮

Following





Photo by [Jei Lee](#) on [Unsplash](#)

[Read this for free](#)

AI coding agents are great at code. They're terrible at staying visually consistent.

Five pages into my new app, I had five different design systems. Every page looked fine on its own. Clean layouts, sensible colors, modern feel. But the buttons didn't match. The grays drifted. The spacing had no shared logic. Individually polished, collectively incoherent.

My CLAUDE.md told the agent how to code. It didn't tell the agent how to *design*. And the agent never asked.

The 4 Lines Every CLAUDE.md Needs

What Karpathy diagnosed, what 60,000 developers bookmarked, and why behavioral constraints beat feature checklists

levelup.gitconnected.com

Every designer and developer using AI to build UI hits the same wall. The code works. The pages don't match.

On April 21, Google open-sourced a spec called DESIGN.md — a markdown format for describing a design system in a way AI agents read natively. No dependencies, no API, no build step. Just plain text.

Then VoltAgent's awesome-design-md took that format and built a collection of 423 brand design systems on top of it. 70,000 GitHub stars in

five weeks.

70,000 developers chose a markdown file over their Figma plugins, design token pipelines, and screenshot workflows. That number tells you where the real bottleneck in AI-generated UI actually is.

In this article:

- The three failure modes that make AI-generated UI look broken
- The 9 sections of a DESIGN.md, mapped to the specific failures each one prevents
- The Design Gap: why models that write perfect code produce terrible UI
- How to set up your file (three paths, all under 5 minutes)
- Where 9 sections aren't enough (honest limitations)
- The Visual Bottleneck: what 70,000 stars tell you about where AI-assisted design is heading

The three failure modes of AI-generated UI — Diagram by Author

What the Community Actually Diagnosed

The models aren't bad at design. They're bad at design memory.

Ask Claude Code to build a login page and it'll produce something clean. Ask it to build the dashboard next, and it starts from scratch. New color choices, new spacing, new button styles. Not because the first ones were wrong, but because the agent doesn't remember them. Every prompt is a blank slate.

This isn't a single problem. It's three distinct failure modes that compound across a project.

The Bootstrap Default. Without any design context, agents converge on the same look: white background, blue primary buttons, gray text, 4px border-radius, system font stack. It's not ugly. It's generic. Your app looks like every other AI-generated app. Your brand disappears into Tailwind defaults.

The Color Roulette. The agent picks a reasonable color for each element, but the picks don't form a system. Red shows up as a decorative accent on one page and an error indicator on the next. The primary blue shifts between #3b82f6 and #2563eb depending on which prompt generated which component. Each choice is defensible in isolation. Together they create visual noise that makes users feel, without being able to articulate why, that something is off.

The Style Drift. This is the one developers complain about most. The same agent, in different conversations, produces different designs for the same

app. Rounded corners in one session, square corners in the next. 8px spacing on Monday, 16px on Tuesday. Even the same button looks different across different conversations. The agent isn't being inconsistent on purpose. It just has no persistent reference for what "your design" looks like.

These aren't bugs. The models are doing exactly what you'd expect from a system with no persistent design context.

They generate reasonable defaults, and reasonable defaults drift.

The question is what the minimum amount of design context looks like. The community's answer turned out to be nine sections of markdown.

70,000 Stars on a Text File

The fix for inconsistent AI design isn't a better model. It's a better brief.

Designers have known this for decades. Every design system, from IBM's Carbon to Shopify's Polaris, encodes the same core idea: **consistency comes from shared constraints, not shared taste.**

Color roles, not color preferences. Spacing scales, not spacing guesses. A typography hierarchy that every page obeys, not font choices that feel right

in the moment.

The problem was that none of these systems spoke a language AI agents could read. Design tokens live in JSON pipelines. Brand guidelines live in Figma. Style decisions live in a designer's head. When you ask an AI agent to build a page, it can't access any of that. It starts from zero.

DESIGN.md solves this by encoding a design system in the one format LLMs already understand: markdown. Google developed the format inside Stitch, their AI design tool, and open-sourced the spec on April 21 under Apache 2.0. Their stated principle: “constrained AI produces more consistent output than unconstrained AI.”

DESIGN.md combines machine-readable tokens with human-readable intent — Diagram by Author

The format captures both *values* and *intent*. Tokens in YAML give agents exact hex codes and spacing numbers. Prose in markdown explains why those values exist and when to break the rules. A JSON token file says "primary": "#635bff" . A DESIGN.md says "Blurple for primary actions and trust signals, never decorative." The agent gets the what *and* the judgment context.

VoltAgent's awesome-design-md took this format and ran with it. 423 brand design systems extracted and formatted, from Stripe to Notion to Airbnb. 70,000 stars in five weeks. Drop one into your project root, next to your CLAUDE.md, and the agent generates UI that matches the brand.

The insight isn't the repo. It's the principle underneath it: **the same constraints that make human design teams consistent make AI agents consistent too.** Color roles. Spacing scales. Typography hierarchies. Do's and don'ts.

These aren't new ideas. They're old ideas in a new format.

The 9 Sections

Here are the nine sections, with Stripe's DESIGN.md as the running example. Each one exists because without it, agents produce a specific, predictable failure.

The sections fall into three clusters. The first three set the brand. The next three build the components. The last three keep everything consistent.

The 9 sections in three clusters: Foundation, Components, Guardrails — Diagram by Author

The Foundation: Setting the Brand

1. Visual Theme & Atmosphere describes the *feel* of the design, not the

values. Stripe's file opens with "technical and luxurious, precise and warm." Without this, the agent has no north star. It defaults to generic clean-modern because that's the safest bet. This section is what separates "a dashboard" from "a Stripe dashboard."

2. Color Palette & Roles is where Color Roulette dies. The section doesn't just list hex values. It assigns semantic roles. Stripe's file maps 30+ colors across seven categories: primary, accent, interactive states, neutral scale, surfaces, borders, shadows. The key word is *roles*. A JSON design token file gives you "primary": "#635bff". A DESIGN.md gives you "Blurple (#635bff) for primary actions and trust signals. Never decorative, never on error states." The agent doesn't just know the color. It knows *when to use it and when not to*.

Color values alone cause Color Roulette. Color roles prevent it. — Diagram by Author

This is the section worth spending time on. Every color gets a job description, not just a hex code. Error states use Ruby (#ea2261). Success uses Emerald. Backgrounds use carefully graduated neutrals. When the agent reaches for a color, it has a reason to pick one over another instead of rolling dice.

3. Typography Rules prevents the font chaos that makes AI-generated pages feel amateur. The section defines a full hierarchy: font families, 14+ size

levels from Display Hero (56px) down to Nano (8px), weights, line-heights, letter-spacing, even OpenType features. Stripe's file specifies weight 300 as the signature headline weight and explicitly says "never use 700 for headlines." Without this, agents default to system fonts and arbitrary sizes that have no relationship to each other.

The Components: Building the UI

4. Component Stylings gives the agent a vocabulary of parts. Button variants with hover, active, disabled, and loading states. Card specs with border-radius, shadow, and border values. Input fields, badges, navigation patterns. Without this section, every button is a blue rectangle with no states, every card is a white box with `box-shadow: 0 2px 4px rgba(0,0,0,0.1)`, and nothing feels interactive.

5. Layout Principles defines the spatial rhythm. Base spacing unit (Stripe uses 8px), max content width, grid structure, border-radius scale, whitespace philosophy. This is what prevents the cramped forms and inconsistent padding that plague AI-generated layouts. The agent learns that spacing isn't arbitrary. It follows a scale.

6. Depth & Elevation prevents the flat, generic shadow that agents paste on everything. Stripe's file defines a five-level shadow system from Flat to Deep, using blue-gray tinted shadows (never neutral gray). The instruction "blue-tinted shadows, never neutral" is exactly the kind of constraint that agents reliably follow when you spell it out and reliably ignore when you don't.

The Guardrails: Keeping It Consistent

7. Do's and Don'ts is the anti-hallucination layer. This is where you tell the agent what *not* to do, because agents are optimized to produce plausible output, and plausible output often violates brand rules in subtle ways. Stripe's file has eight do's and seven don'ts. "Never use border-radius above 8px." "Never use pill-shaped buttons." "Never use heavy font weights for headlines." These are exactly the kind of "reasonable" choices an agent would make without being told otherwise.

This section matters more than most people expect. **Agents don't hallucinate wildly on design. They hallucinate conservatively.** They reach for safe, familiar patterns, and those safe patterns are often wrong for your specific brand.

Pill-shaped buttons are a perfectly reasonable default. They're just not Stripe. The Do's and Don'ts section is how you encode the difference between "a good design" and "your design."

8. Responsive Behavior defines breakpoints with pixel ranges, collapsing strategies per component type, and touch-target adjustments. Without it, mobile output is desktop squeezed into a phone. Navigation doesn't collapse. Typography doesn't scale. Cards that looked elegant at 1200px become cramped at 375px. This section tells the agent that mobile isn't smaller desktop. It's a different design context.

9. Agent Prompt Guide solves the cold-start problem. It provides quick-

reference color codes, ready-to-use component prompts, and iteration checklists that the agent can slot directly into its working memory. Without this, every new conversation starts from zero, which is exactly how Style Drift begins. This section is the persistence layer that markdown files were missing.

Code quality rose with tooling. Design quality stayed flat — until DESIGN.md. — Diagram by Author

The Design Gap

Here's what's strange about the current state of AI coding. The models can reason about TypeScript generics, infer complex type hierarchies, and refactor a 500-line function into clean modules. But they can't keep a button the same shade of blue across two pages.

This isn't because design is harder than code. It's because the tooling invested everything in one and almost nothing in the other.

Code has CLAUDE.md, .cursorrules, copilot-instructions.md, linters, formatters, type checkers, CI pipelines. Every layer of the stack tells the agent what "correct code" looks like.

Design has nothing. No equivalent file. No guardrails. No persistent context. The agent generates each page in a vacuum and hopes for the best.

That's the Design Gap: AI coding tools developed deep infrastructure for code consistency and zero infrastructure for visual consistency.

The models are equally capable at both. The tooling isn't.

Google's decision to open-source the DESIGN.md spec is an acknowledgment that this gap is structural, not temporary. It won't close by making models smarter. GPT-5 writes better code than GPT-4, but it generates equally inconsistent UI without design context. **The gap is an**

input problem. The models need design briefs the same way they need coding guidelines.

The reason markdown closes this gap where other formats didn't comes down to how agents process information. Design tokens in JSON are 40–55% more expensive in tokens for the same semantic content. And they only carry values, not reasoning. A JSON file can't say “never use this color decoratively.” A markdown file can, and agents follow prose instructions more reliably than they follow structured data.

This is also why screenshot-based approaches don't scale. Pasting a reference image into a prompt works for a single component, but it gives the agent pixels without principles. It can try to match the visual, but it has no understanding of *why* the design looks that way. Change the screen size, add a new component type, and the agent is guessing again. **DESIGN.md** gives the agent the reasoning, not just the reference.

What to Actually Put in Your Project

Three paths, depending on where you're starting.

Path A: Pick a brand from the collection.

The fastest option. Browse [awesome-design-md](#), find a brand that matches your aesthetic, and drop it in:

```
# Example: use Stripe's design system
curl -o DESIGN.md https://raw.githubusercontent.com/VoltAgent/awesome-design-md
```

423 brands available. Stripe, Linear, Notion, Vercel, Airbnb, Spotify, and hundreds more. This works surprisingly well for prototypes and side projects where you want something polished without hiring a designer.

Path B: Generate from your existing site.

If you already have a live product, Google Stitch can extract a DESIGN.md from your URL automatically. It's free, no account required. Stitch reads your site's CSS, identifies the design system underneath, and outputs a structured DESIGN.md you can drop into your repo.

The extracted file won't be perfect. You'll want to review the Do's and Don'ts section especially, since Stitch infers rules it can observe but can't read your designer's mind. Treat it as a strong starting point, not a finished spec.

Path C: Write your own.

You don't need all nine sections on day one. Start with four:

1. **Visual Theme & Atmosphere** — two sentences describing the feel
2. **Color Palette & Roles** — your primary, accent, error, and neutral colors

with semantic roles

3. **Component Stylings** — button variants and card specs at minimum
4. **Do's and Don'ts** — five rules the agent should never break

These four sections cover the highest-impact failure modes. Add Typography, Layout, and the rest as you hit inconsistencies. A partial DESIGN.md is better than none.

Three paths to your DESIGN.md, all under 15 minutes — Diagram by Author

Where to put it: Project root, next to your CLAUDE.md or .cursorrules . The agent picks it up automatically.

DESIGN.md sits next to CLAUDE.md in your project root — Diagram by Author

Where 9 Sections Aren't Enough

DESIGN.md solves the context problem. It doesn't solve everything.

Token cost is real. A full DESIGN.md consumes roughly 30,000 tokens per query. That's context window space your agent can't use for code reasoning. JSON design tokens carry the same values in roughly 80% fewer tokens, but

lose the intent layer. For large projects with tight context budgets, you may need to trim your DESIGN.md to the four essential sections (Theme, Colors, Components, Do's/Don'ts) and leave the rest as a reference doc the agent consults on demand.

The token tradeoff: start with 4 sections, expand as needed — Diagram by Author

Design drift hasn't been solved. A DESIGN.md extracted from a live site is a snapshot. When the site's design evolves, the file goes stale. There's no auto-sync mechanism between your live CSS and your DESIGN.md. Someone has to manually update the file, and in practice, that update lags. This is the same version-control challenge design tokens have always had, just in a new format.

There's no runtime enforcement. The agent reads the spec and *tries* to follow it. But nothing stops it from outputting a `border-radius: 16px` when your Do's and Don'ts say "never above 8px." DESIGN.md is guidance, not a linter. If you need hard enforcement, you still need CSS linting rules or a design review step on top.

Team scaling is a coordination problem. One developer's DESIGN.md is easy. Getting a team of 20 to maintain a markdown file alongside their Figma workflow is a different challenge entirely. Most design teams work Figma-first, not markdown-first. Until there's better tooling to keep DESIGN.md and Figma in sync, teams will face a two-source-of-truth problem.

The spec is still alpha. Google's official specification is a draft. Missing pieces include animation and motion tokens, icon standards, and accessibility checking. The community is filling gaps fast, but if you need motion design or WCAG validation, you'll need to add those constraints yourself.

One honest note on the 70,000 stars: they're a signal of resonance, not proof of efficacy. We don't have controlled studies showing exactly how much DESIGN.md improves output consistency versus a well-prompted agent with no file. The community consensus is strong, but it's anecdotal. I'd want to see formal benchmarks before calling this settled science.

CLAUDE.md solved code consistency. DESIGN.md solves design consistency. Same pattern. — Diagram by Author

The Visual Bottleneck

70,000 stars on a collection of markdown files tells you something the product announcements don't: the bottleneck in AI-assisted development was never the model's ability to write CSS. It was design context.

The models can generate any visual style you want. They can produce Stripe's precision, Linear's minimalism, Notion's warmth. But only if you tell them which one. Without a persistent design brief, every page is a fresh guess. And fresh guesses drift.

This is the same pattern we saw with code behavior.

- CLAUDE.md's four behavioral lines solved the coding bottleneck not by making the model smarter, but by giving it constraints.
- DESIGN.md's nine sections do the same thing for the visual layer. Both are plain text.

Both beat complex toolchains. Both work because they match how agents actually consume information: structured prose that carries values and intent in a format the model was trained on.

The design toolchain arms race — Figma plugins, token pipelines, Stitch workflows — is solving the capability layer.

But the binding constraint was always the context layer. A markdown file in your project root. Nine sections. No dependencies.

Google open-sourcing the spec signals that this format is heading toward a standard. **The question isn't whether AI agents will read design briefs. They already do. The question is whether yours will have one.**

If you want to start, the repo is at github.com/VoltAgent/awesome-design-md. Pick a brand, drop the file, and build a page. The difference shows up on the first render.

Harness Engineering: What Every AI Engineer Needs to Know in 2026

Three camps, three architectures — and what Opus 4.7 just proved about all of them

ai.gopubby.com

OpenAI Symphony vs Claude Managed Agents vs CrewAI: Which Agent Orchestration Pattern Wins

Three architectures benchmarked. \$617K in annual cost difference. One Pareto-optimal winner.

ai.gopubby.com

Before you go! 🧐

If you liked my story and you want to support me:

1. Throw some Medium love 💕 (claps, comments and highlights), your support means the world to me. 🙌
2. Follow me on Medium and subscribe to get my latest article 🙌

This story is published on Generative AI. Connect with us on LinkedIn and follow Zeniteq to stay in the loop with the latest AI stories.

Subscribe to our newsletter and YouTube channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

Technology

Artificial Intelligence

Programming



Published in Generative AI

87K followers · Last published 6 hours ago

Following

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com



Written by Yanli Liu

6.1K followers · 42 following

Follow

Daytime finance practitioner based in Luxembourg, seasoned coder, and passionate about anything AI-related!

